# Multithreading

# Introduction

- Multithreading is a fundamental feature in Java that allows concurrent execution of two or more threads. A thread is a lightweight process and is the smallest unit of CPU execution. Java provides a built-in thread model that simplifies multithreading implementation.
- Java’s multithreading model is designed to **execute multiple tasks simultaneously** within a single program

# Creating Thread in Java

- By extending Thread class, we can override the run() method.

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running...");
    }
}

public class ThreadExample {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();  // Start the thread
    }
}
```

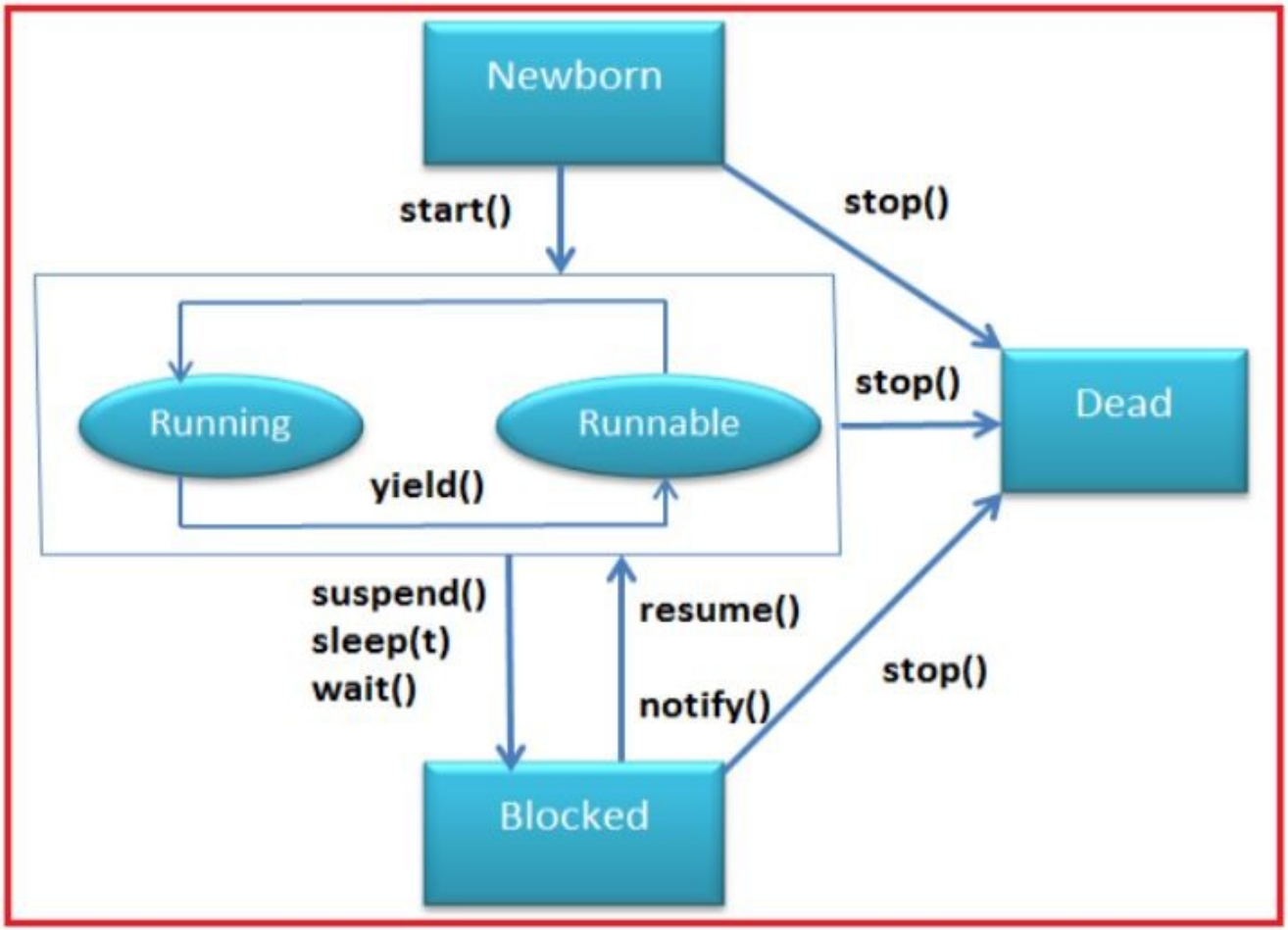
# Implementing the Runnable Interface

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread is running...");
    }
}

public class RunnableExample {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyRunnable());
        t1.start();
    }
}
```

- A class can extend another class while still implementing Runnable.

# Life Cycle of a Thread



## 1. New (Created) State

- A thread is in the **new state** when it is created using either Thread class or Runnable interface.
- The thread is **not yet started**.
- It remains in this state until the start() method is called.

## 2. Runnable State

- When the start() method is called, the thread moves to the **runnable** state.
- The thread is **ready to run** but waiting for CPU time.
- The **scheduler decides** which thread will execute based on priorities.

- A program implementing multithreading acquires a fixed slice of time to each individual thread.
- Each and every thread runs for a short span of time and when that allocated time slice is over, the thread voluntarily gives up the CPU to the other thread, so that the other threads can also run for their slice of time.
- Whenever  a scenario with a high priority thread comes , all those threads that are willing to run, waiting for their turn to run, lie in the runnable state. In the runnable state, there is a queue where the threads lie.

3. **Running:**

The thread enters the **running** state when the CPU starts executing it. This state depends on the thread scheduler, which assigns CPU time to the thread.

4. **Blocked or Waiting:** Whenever a thread is inactive for a span of time (not permanently) then, either the thread is in the blocked state or is in the waiting state.

- For example, a thread (let's say its name is A) may want to print some data from the printer. However, at the same time, the other thread (let's say its name is B) is using the printer to print some data.
- Therefore, thread A has to wait for thread B to use the printer. Thus, thread A is in the blocked state. A thread in the blocked state is unable to perform any execution and thus never consume any cycle of the Central Processing Unit (CPU).
- Hence, we can say that thread A remains idle until the thread scheduler reactivates thread A, which is in the waiting or blocked state.

- Terminated (Dead)

A thread enters the terminated state when it completes execution or is forcibly stopped. Once terminated, it cannot be restarted.

# Thread Priority

- In Java, each thread has a priority that helps determine the order in which threads are scheduled for execution.
- Thread priorities are represented as integers ranging from Thread.MIN_PRIORITY (1) to Thread.MAX_PRIORITY (10), with the default priority being Thread.NORM_PRIORITY (5).

How Thread Priority Works

- Higher-priority threads are generally executed before lower-priority ones, but thread scheduling is managed by the JVM and operating system, so priority does not guarantee execution order.
- The setPriority(int priority) method is used to change a thread's priority.
- The getPriority() method retrieves the current priority of a thread.

# Thread Exceptions in Java

- In Java, exceptions in threads occur when a thread encounters an error during execution, such as InterruptedException, IllegalThreadStateException, or RuntimeException.
- If an exception occurs inside a thread and is not handled properly, it may cause the thread to terminate unexpectedly.
- Common Thread Exceptions:
- InterruptedException – Thrown when a sleeping or waiting thread is interrupted.
- IllegalThreadStateException – Occurs when an operation is performed at an inappropriate time, e.g., starting a thread that is already running.
- RuntimeException & Errors – If an uncaught exception occurs in a thread, the JVM terminates that thread.

# Synchronization

- When multiple threads access a shared resource **simultaneously**, unexpected behavior can occur due to **race conditions** (when multiple threads modify shared data at the same time). Synchronization ensures:
- **Data consistency**: Prevents inconsistent results.
- **Thread safety**: Ensures only one thread accesses a critical section at a time.

- Synchronized keyword used.
- Java creates a monitor and hands it over to the thread that calls the method first time
- As long as the thread holds the monitor, no other thread can enter the synchronized section of the code.
- A monitor is like a key and the thread that holds the key can only open the lock
- When the thread completed the work of using synchronized code, it will hand over the monitor to the next thread ready to use the same resource

## 1. Synchronized Methods

- A method can be declared synchronized to ensure that only one thread can execute it at a time.

## 2. Synchronized Blocks

- Instead of synchronizing the entire method, you can synchronize a specific block inside it. This improves performance by locking only the critical section.